

Exercices avancés : Programmation Orientée Objet (POO)

Ces exercices visent à renforcer la compréhension des concepts de POO en Python : - Classes et objets - Constructeurs et méthodes - Encapsulation et gestion des attributs - Héritage et polymorphisme - Organisation en modules

Exercice 1 : Gestion de compte bancaire sécurisé

Énoncé :

Créez une classe `CompteBancaire` qui permet de : - Créer un compte avec un solde initial - Faire des dépôts et retraits - Vérifier le solde - Empêcher les retraits supérieurs au solde

Ajoutez également : - Une méthode pour appliquer des intérêts (`appliquer_interets(taux)`) - Des vérifications robustes - Des noms d'utilisateurs uniques (ex. empêcher deux comptes avec le même nom)

Utilisez : - Attributs privés - Getters/setters ou `@property` - Méthodes de validation

```
class CompteBancaire:
    """
    Représente un compte bancaire avec gestion du solde, dépôts et retraits.

    Attributes:
        titulaire (str): Nom du titulaire du compte
        solde (float): Solde actuel (en lecture seule)
    """

    titulaires_de_comptes = set() # Ensemble des titulaires existants

    def __init__(self, titulaire, solde_initial=0):
        """
```

```

Initialise un nouveau compte bancaire.

Args:
    titulaire (str): Nom du titulaire du compte
    solde_initial (float): Solde initial du compte

Raises:
    ValueError: Si le titulaire existe déjà
"""
if titulaire in CompteBancaire.titulaires_de_comptes:
    raise ValueError(f"Le titulaire '{titulaire}' existe déjà.")
self._titulaire = titulaire
self._solde = max(float(solde_initial), 0)
CompteBancaire.titulaires_de_comptes.add(titulaire)

@property
def titulaire(self):
    """str: Nom du titulaire (lecture seule)"""
    return self._titulaire

@property
def solde(self):
    """float: Solde actuel du compte (lecture seule)"""
    return self._solde

def depot(self, montant):
    """
    Ajoute un montant au solde.

    Args:
        montant (float): Montant à ajouter

    Raises:
        ValueError: Si le montant est négatif ou nul
    """
    if montant <= 0:
        raise ValueError('Le montant doit être positif')
    self._solde += montant

def retrait(self, montant):
    """
    Retire un montant au solde.

```

```

    Args:
        montant (float): Montant à retirer

    Raises:
        ValueError: Si le montant est inférieur ou égal à zéro
    """
    if montant <= 0:
        raise ValueError('Le montant doit être positif')
    if montant > self._solde:
        raise ValueError('Solde insuffisant')
    self._solde -= montant

def appliquer_interets(self, taux):
    """
    Applique un taux d'intérêt au solde.

    Args:
        taux (float): Taux à appliquer (ex. 0.05 pour 5%)

    Raises:
        ValueError: Si le taux n'est pas entre 0 et 1
    """
    if not 0 <= taux <= 1:
        raise ValueError('Le taux doit être entre 0 et 1')
    self._solde *= (1 + taux)

def afficher_solde(self):
    """Affiche le solde actuel."""
    print(f"Solde actuel : {self._solde:.2f} €")

```

```

try:
    compte_d_Alice = CompteBancaire('Alice', 100)
    compte_de_Bob = CompteBancaire('Bob', 50)
    compte_autre_Alice = CompteBancaire('Alice') # Devrait lever une erreur
except ValueError as e:
    print(e)

compte_d_Alice.depot(50)
compte_d_Alice.retrait(30)
compte_d_Alice.appliquer_interets(0.05)
compte_d_Alice.afficher_solde()

```

Le titulaire 'Alice' existe déjà.
Solde actuel : 126.00 €

```
try:
    compte_d_Alice.retrait(150) # Devrait lever une erreur
except ValueError as e:
    print(e)
```

Solde insuffisant

Exercice 2 : Héritage – Formes géométriques

Énoncé :

Implémentez une hiérarchie de classes représentant différentes formes géométriques : - Classe mère : `Forme` - Sous-classes : `Rectangle`, `Cercle`

Chaque forme doit avoir : - Une méthode `.aire()` - Une méthode `.perimetre()`

Bonus : - Comparez les aires via `__lt__`, `__eq__` - Affichez chaque objet avec `__str__`

Objectif : Pratiquer l'héritage, le polymorphisme et la réutilisation de code.

```
from abc import ABC, abstractmethod
import math

class Forme(ABC):
    """
    Classe abstraite représentant une forme géométrique.
    Ne peut pas être instanciée directement.
    """

    @abstractmethod
    def aire(self):
        """Calcule l'aire de la forme."""
        pass

    @abstractmethod
    def perimetre(self):
        """Calcule le périmètre de la forme."""
        pass
```

```

class Rectangle(Forme):
    """
    Représente un rectangle.

    Attributes:
        longueur (float): Longueur du rectangle
        largeur (float): Largeur du rectangle
    """

    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    @property
    def longueur(self):
        """float: Longueur du rectangle"""
        return self._longueur

    @longueur.setter
    def longueur(self, valeur):
        if valeur <= 0:
            raise ValueError("La longueur doit être positive")
        self._longueur = float(valeur)

    @property
    def largeur(self):
        """float: Largeur du rectangle"""
        return self._largeur

    @largeur.setter
    def largeur(self, valeur):
        if valeur <= 0:
            raise ValueError("La largeur doit être positive")
        self._largeur = float(valeur)

    def aire(self):
        """Calcule l'aire du rectangle."""
        return self._longueur * self._largeur

    def perimetre(self):
        """Calcule le périmètre du rectangle."""
        return 2 * (self._longueur + self._largeur)

```

```

class Cercle(Forme):
    """
    Représente un cercle.

    Attributes:
        rayon (float): Rayon du cercle
    """

    def __init__(self, rayon):
        self.rayon = rayon

    @property
    def rayon(self):
        """float: Rayon du cercle"""
        return self._rayon

    @rayon.setter
    def rayon(self, valeur):
        if valeur <= 0:
            raise ValueError("Le rayon doit être positif")
        self._rayon = float(valeur)

    def aire(self):
        """Calcule l'aire du cercle."""
        return math.pi * self._rayon ** 2

    def perimetre(self):
        """Calcule le périmètre du cercle."""
        return 2 * math.pi * self._rayon

```

```

# Test des formes géométriques
rect = Rectangle(4, 5)
print(rect.aire(), rect.perimetre())

cercle = Cercle(3)
print(f'{cercle.aire():.2f}, {cercle.perimetre():.2f}')

```

```

20.0 18.0
28.27, 18.85

```

Exercice 3 : Système de gestion de bibliothèque

Énoncé :

Concevez un système orienté objet pour une bibliothèque, avec les classes suivantes : - **Livre** : titre, auteur, isbn, disponibilité - **Membre** : nom, identifiant, livres empruntés - **Bibliotheque** : liste de livres, méthodes pour emprunter/rendre/lister

Fonctionnalités attendues : - Recherche par titre ou auteur - Emprunt/retour de livre - Liste des livres disponibles - Sauvegarde/restauration depuis un fichier JSON

Utilisez : - POO complète - Modules standards (json) - Organisation en fichiers (si temps)

```
class Livre:
    """
    Représente un livre avec titre, auteur et disponibilité.

    Attributes:
        titre (str): Titre du livre
        auteur (str): Auteur du livre
        isbn (str): ISBN du livre
    """

    def __init__(self, titre, auteur, isbn):
        self.titre = titre
        self.auteur = auteur
        self.isbn = isbn
        self._disponible = True

    @property
    def disponible(self):
        """bool: Indique si le livre est disponible"""
        return self._disponible

    def emprunter(self):
        """Rend le livre indisponible si possible."""
        if not self._disponible:
            return False
        self._disponible = False
        return True

    def rendre(self):
        """Rend le livre à nouveau disponible."""
        self._disponible = True
```

```

class Membre:
    """
    Représente un membre de la bibliothèque.

    Attributes:
        nom (str): Nom du membre
    """

    def __init__(self, nom):
        self.nom = nom
        self.livres_empruntes = []

    @property
    def livres(self):
        """list: Liste des livres empruntés"""
        return self.livres_empruntes

    def emprunter(self, livre):
        """
        Emprunte un livre.

        Args:
            livre (Livre): Le livre à emprunter

        Returns:
            bool: True si réussi, False sinon
        """
        if livre.emprunter():
            self.livres_empruntes.append(livre)
            return True
        return False

    def rendre(self, livre):
        """
        Rend un livre emprunté.

        Args:
            livre (Livre): Le livre à rendre
        """
        if livre in self.livres_empruntes:
            livre.rendre()

```

```

        self.livres_empruntes.remove(livre)

def afficher_livres(self):
    """Affiche les livres actuellement empruntés."""
    if not self.livres_empruntes:
        print("Aucun livre emprunté.")
    else:
        for livre in self.livres_empruntes:
            print(f"- {livre.titre} - {livre.auteur}")

```

```

import json

class Bibliotheque:
    """
    Représente une bibliothèque gérant une collection de livres.

    Attributes:
        livres (list): Liste des livres disponibles dans la bibliothèque
    """

    def __init__(self):
        """Initialise une bibliothèque vide."""
        self.livres = []

    def ajouter_livre(self, livre):
        """
        Ajoute un livre à la collection.

        Args:
            livre (Livre): Le livre à ajouter
        """
        self.livres.append(livre)

    def chercher_par_titre(self, titre):
        """
        Cherche tous les livres contenant le titre donné.

        Args:
            titre (str): Titre à rechercher

        Returns:
            list: Liste des livres trouvés

```

```

    """
    return [livre for livre in self.livres if titre.lower() in livre.titre.lower()]

def chercher_par_auteur(self, auteur):
    """
    Cherche tous les livres dont l'auteur correspond.

    Args:
        auteur (str): Auteur à rechercher

    Returns:
        list: Liste des livres trouvés
    """
    return [livre for livre in self.livres if auteur.lower() in livre.auteur.lower()]

def emprunter_livre(self, isbn, membre):
    """
    Permet à un membre d'emprunter un livre.

    Args:
        isbn (str): ISBN du livre à emprunter
        membre (Membre): Membre qui emprunte le livre

    Returns:
        bool: True si succès, False sinon
    """
    for livre in self.livres:
        if livre.isbn == isbn and livre.disponible:
            if membre.emprunter(livre):
                return True
    return False

def rendre_livre(self, isbn, membre):
    """
    Permet à un membre de rendre un livre.

    Args:
        isbn (str): ISBN du livre à rendre
        membre (Membre): Membre qui rend le livre

    Returns:
        bool: True si succès, False sinon

```

```

    """
    for livre in self.livres:
        if livre.isbn == isbn:
            membre.rendre(livre)
            livre.rendre()
            return True
    return False

def lister_disponibles(self):
    """
    Liste tous les livres actuellement disponibles.

    Returns:
        list: Liste des livres disponibles
    """
    return [livre for livre in self.livres if livre.disponible]

def sauvegarder(self, chemin):
    """
    Sauvegarde la liste des livres dans un fichier JSON.

    Args:
        chemin (str): Chemin vers le fichier JSON
    """
    donnees = []
    for livre in self.livres:
        donnees.append({
            'titre': livre.titre,
            'auteur': livre.auteur,
            'isbn': livre.isbn,
            'disponible': livre.disponible
        })
    with open(chemin, 'w') as f:
        json.dump(donnees, f, indent=2)

def charger(self, chemin):
    """
    Charge la liste des livres depuis un fichier JSON.

    Args:
        chemin (str): Chemin vers le fichier JSON
    """

```

```

try:
    with open(chemin, 'r') as f:
        donnees = json.load(f)
    for item in donnees:
        livre = Livre(item['titre'], item['auteur'], item['isbn'])
        if not item['disponible']:
            livre._disponible = False # On force la disponibilité
        self.livres.append(livre)
except FileNotFoundError:
    print("Fichier introuvable. Début avec une bibliothèque vide.")

```

```

# Création de quelques livres
livre1 = Livre('Le Seigneur des Anneaux', 'J.R.R. Tolkien', '978-3-16-148410-0')
livre2 = Livre('Le Hobbit', 'J.R.R. Tolkien', '978-3-16-148411-7')
livre3 = Livre('Harry Potter à l\'école des sorciers', 'J.K. Rowling', '978-2-07-053426-8')

# Création d'un membre
membre1 = Membre('Alice')

# Création de la bibliothèque
biblio = Bibliotheque()
biblio.ajouter_livre(livre1)
biblio.ajouter_livre(livre2)
biblio.ajouter_livre(livre3)

# Recherche par auteur
print("Livres de Tolkien :")
for livre in biblio.chercher_par_auteur('Tolkien'):
    print(f"- {livre.titre}")

# Emprunt d'un livre
print("\nEmprunt du livre 'Le Hobbit' :")
if biblio.emprunter_livre('978-3-16-148411-7', membre1):
    print(" Emprunt réussi !")
else:
    print(" Emprunt échoué.")

# Affichage des livres empruntés
print("\nLivres empruntés par Alice :")
membre1.afficher_livres()

# Sauvegarde dans un fichier

```

```

biblio.sauvegarder('bibliotheque.json')

# Chargement depuis le fichier
nouvelle_biblio = Bibliotheque()
nouvelle_biblio.charger('bibliotheque.json')

print("\nNombre de livres dans la nouvelle instance après chargement :", len(nouvelle_biblio

```

Livres de Tolkien :

- Le Seigneur des Anneaux
- Le Hobbit

Emprunt du livre 'Le Hobbit' :

Emprunt réussi !

Livres empruntés par Alice :

- Le Hobbit - J.R.R. Tolkien

Nombre de livres dans la nouvelle instance après chargement : 3

Exercice 4 : Modélisation d'un système de véhicules

Énoncé :

Modélisez une hiérarchie de véhicules : - Classe mère : **Vehicule** - Sous-classes : **Voiture**, **Moto**, **Camion**

Chaque véhicule aura : - Un constructeur - Des attributs : marque, modèle, année, vitesse - Des méthodes : **accelerer()**, **freiner()**, **__str__()**

Bonus : - Implémentez une interface **Roulable** avec **rouler()** implémentée différemment selon le type - Utilisez des classes abstraites (**abc.ABC**, **@abstractmethod**)

Objectif : Maîtriser l'héritage multiple, les méthodes abstraites et le polymorphisme.

```

class Vehicule:
    """
    Classe de base pour tous les véhicules.
    Doit être héritée pour être utilisée.
    """

    def demarrer(self):

```

```

        """Démarré le véhicule."""
        print("Véhicule démarré.")

class Voiture(Vehicule):
    """Représente une voiture."""

    def demarrer(self):
        """Démarré la voiture."""
        print("Voiture prête à rouler.")

class Moto(Vehicule):
    """Représente une moto."""

    def demarrer(self):
        """Démarré la moto."""
        print("Moto prête à rouler.")

class Camion(Vehicule):
    """Représente un camion."""

    def demarrer(self):
        """Démarré le camion."""
        print("Camion lourd prêt à partir.")

vehicules = [Voiture(), Moto(), Camion()]
for v in vehicules:
    v.demarrer()

```

Voiture prête à rouler.
Moto prête à rouler.
Camion lourd prêt à partir.